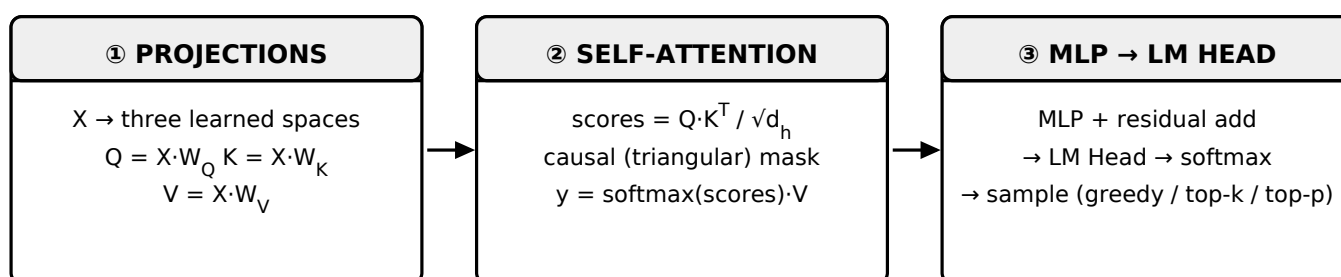


PAGE 1 — AUTOREGRESSIVE INFERENCE & THE COMPUTE BOTTLENECK

1 Autoregressive Generation

- An LLM generates text one token at a time: each new token is appended to the sequence and becomes part of the input for the next step.
- At every step the model re-scores the entire growing history and emits a probability distribution over its full vocabulary.
- Generation loop: Prompt → Transformer → Next token → Append → Repeat (until a stop token or the maximum length).

2 Standard Transformer Block — one layer

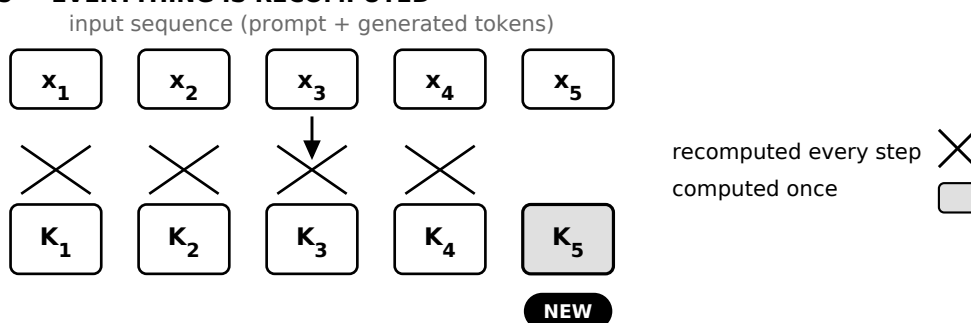


The causal mask sets future positions to $-\infty$, so their softmax weight is 0 — each token attends only to the past.

3 The Problem — Naive Inference

- To produce token x_t , its query Q_t must attend to every previous key and value: $K_{1...t-1}, V_{1...t-1}$.
- A naive implementation recomputes all historical K and V tensors at every decode step.
- But historical K and V never change once computed → recomputing them is pure waste.
- The wasted work grows quadratically with sequence length: $O(T^2 \cdot d)$.

NAIVE DECODE AT STEP $t = 5$ — EVERYTHING IS RECOMPUTED



Every step redoes all prior K, V : step 2 → K_1 · step 3 → K_1, K_2 · step 4 → K_1, K_2, K_3 ...
 Total redundant work = $1 + 2 + 3 + \dots + T = O(T^2)$ → quadratic bottleneck

KEY INSIGHT

Historical Keys and Values are static after their first computation. Recomputing them is pure waste — this is the exact inefficiency the KV cache eliminates.

PAGE 2 — THE KV CACHE SOLUTION & SYSTEM ARCHITECTURE

1 The KV Cache Concept

- Cache (store) the K and V tensors of every previously processed token directly in GPU VRAM — this store is the KV cache.
- Each step, compute K_t and V_t for the new token only, then append the pair to the cache (an $O(1)$ update).
- The new query Q_t attends to the complete cached history K_{all}, V_{all} — no recomputation of old tokens.
- Result: decode-step attention becomes linear in cached context — $O(t \cdot d)$ instead of $O(t^2 \cdot d)$.

2 Diagram 1 — KV Cache: Full System Block Process

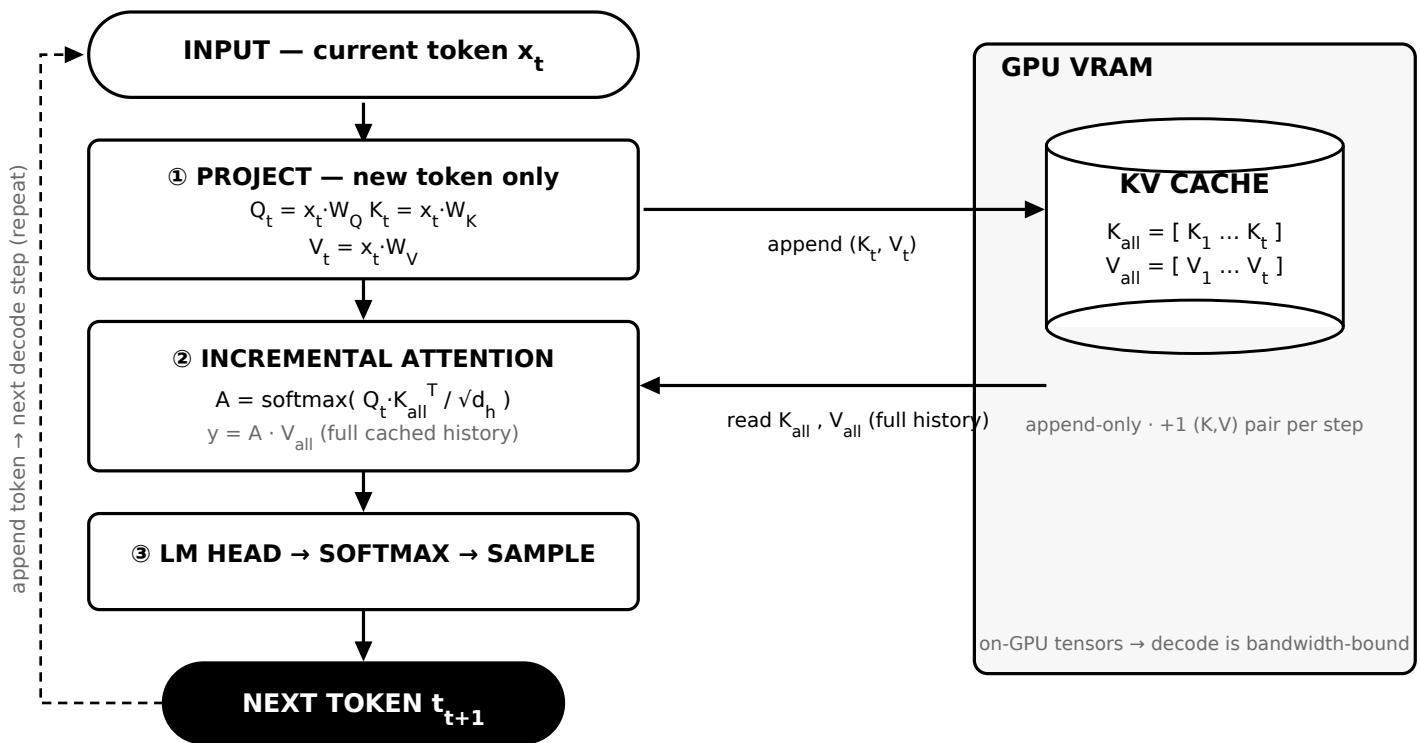


FIGURE 1 — End-to-end decode step with KV cache: project the new token → append to the VRAM cache → attend over the full history → sample the next token.

3 What Happens at Each Decode Step

- 1 Project — compute Q_t, K_t, V_t from the new token only ($x_t \cdot W_{Q/K/V}$).
- 2 Append & attend — add (K_t, V_t) to the VRAM cache, then run attention over the full history: $y = \text{softmax}(Q_t \cdot K_{all}^T / \sqrt{d_h}) \cdot V_{all}$.
- 3 Sample — LM head → softmax → next token t_{t+1} ; append it to the sequence and repeat from step 1.

PAGE 3 — PERFORMANCE PHASES & MEMORY SCALING

1 Prefill vs. Decode — Two Distinct Execution Phases

PHASE	DESCRIPTION	INPUT	MEMORY USAGE	BOTTLENECK
PREFILL	Processes the entire prompt in parallel (single pass).	Full prompt X_{prompt}	Allocates & fills the initial KV cache.	COMPUTE-BOUND — matrix $\times$ matrix multiplies.
DECODE	Generates one token sequentially per step.	Single new token.	Reads cache + appends 1 (K, V) pair.	BANDWIDTH-BOUND — loading cached tensors.

2 Complexity Correction

- Prefill: full-sequence attention is quadratic in the prompt length — $O(T^2 \cdot d)$.
- Decode with KV cache: the new query attends only to the cached history → linear per step — $O(t \cdot d)$.
- So “attention is always quadratic” is a misleading claim for inference: the KV cache makes decode-step compute linear in context length.

3 KV Cache Memory Scaling

$$M_{KV} = 2 \times L \times T \times H_{kv} \times d_{\text{head}} \times \text{bytes}$$

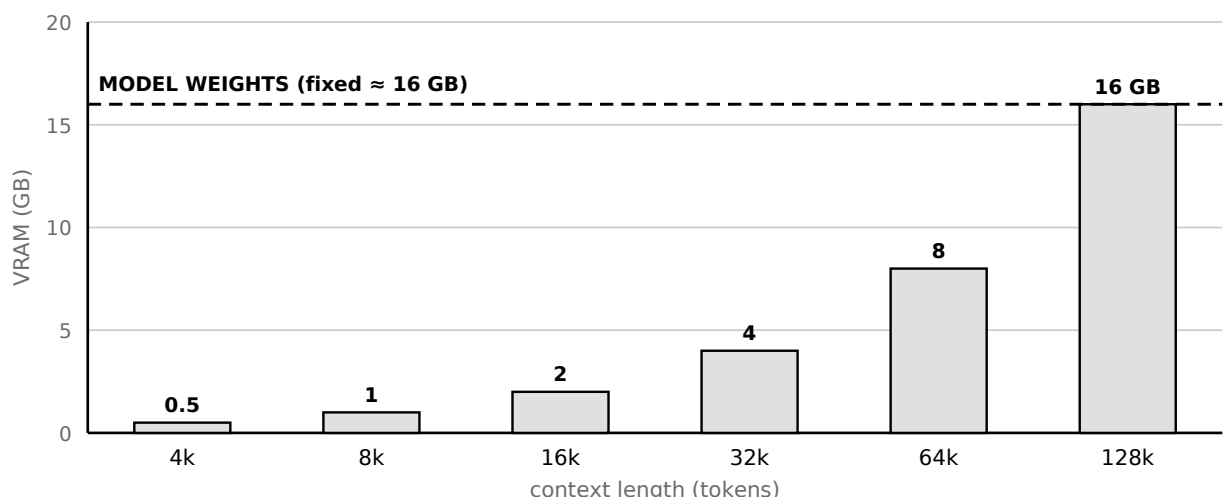
2 — Key + Value matrices

L — Transformer layers

T — Tokens cached = context length

 H_{kv} — Number of KV heads d_{head} — Dimension of each head

bytes — Element size (FP16 = 2 bytes)



□ KV cache ($\propto$ tokens $\times$ batch) - - - Model weights (fixed)

Illustrative: $L=32 \cdot H_{kv}=8 \cdot d_{\text{head}}=128 \cdot \text{FP16} \rightarrow 128 \text{ KB/token}$; 8B model $\approx$ 16 GB weights.

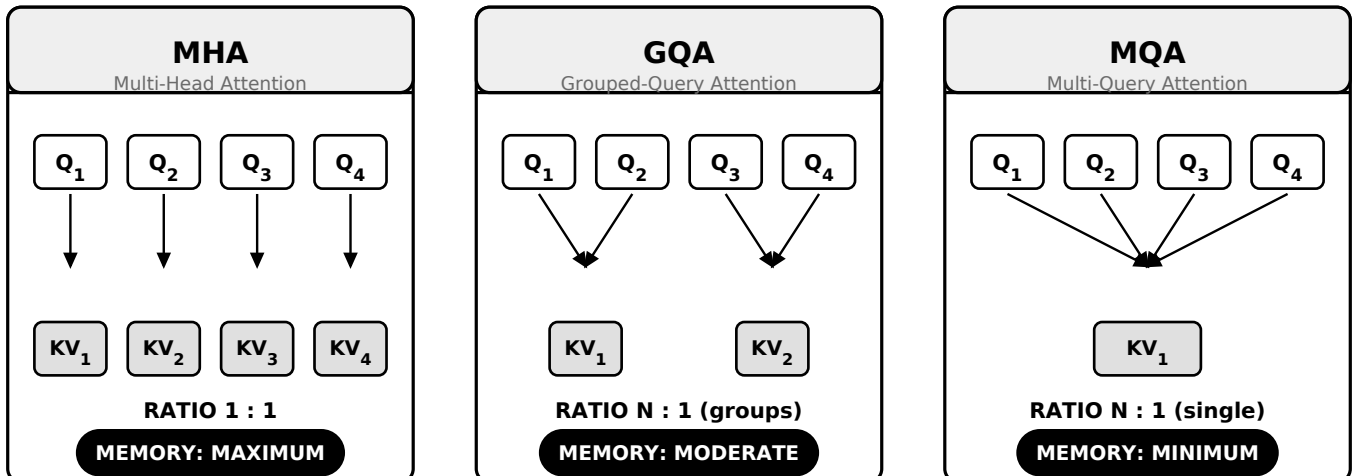
CRUCIAL SCALING

$M_{KV} \propto T$ (and $\propto$ batch size). As context grows 4k $\rightarrow$ 32k $\rightarrow$ 128k, the KV cache overtakes the model weights and becomes the dominant VRAM consumer.

PAGE 4 — TOPOLOGY & ARCHITECTURE SOLUTIONS

To attack the KV-cache memory bottleneck, modern architectures change the ratio of Query (Q) heads to Key-Value (KV) heads:

1 Diagram 2 — Attention Topologies: MHA → GQA → MQA



Q heads (top) send queries to KV heads (bottom). Fewer KV heads $\Rightarrow$ smaller KV cache.

2 Key Architectures

- MHA — Multi-Head Attention. Maximum expressivity, maximum memory pressure — the original Transformer design. Every Query head owns a dedicated KV head (1:1 ratio).
- GQA — Grouped-Query Attention. The sweet spot. Groups of Query heads share a single KV head $\rightarrow$ cuts cache memory while keeping quality and long-context recall. Used by Llama-3, Mistral, Gemma.
- MQA — Multi-Query Attention. All Query heads share one KV head $\rightarrow$ minimum cache, but risks quality loss and degraded recall in very long sequences. Used by PaLM (T5-style).

3 Memory Requirement & Verdict



cache size (per token, per layer)

VERDICT

Modern production models almost universally adopt GQA: it offers the best trade-off between KV-cache memory savings and model quality on long contexts.